

# A Human-Centered Architecture for Instrumented Virtual Reality Training with Digital Twin Connectivity

Saad Joual

FABLAB, HESTIM Casablanca  
ENSAM, Hassan II University of Casablanca  
Casablanca, Morocco  
Saadjawal2003@gmail.com

Sridath Tula

Centre de Recherche en Ingénierie et Management (CERIM)  
HESTIM Engineering & Business School  
Casablanca, Morocco  
s.tula@hestim.ma

**Abstract**—Virtual reality can support industrial training, but a realistic scene alone does not define task order, action validity, or measurable evidence. This paper presents an instrumented platform for a labeling workstation. Implemented in Unity 6 for Meta Quest 3, it represents the task as configurable states coupled to guidance, rule-based validators, and state-level telemetry. A separate read-only ESP32-FastAPI-Unity path demonstrates optional factory supervision without controlling equipment or gating training. Technical evaluation combined requirement-level functional checks with 36 retained session records from 12 students. The logger captured 131 invalid interactions in 31 sessions. Twenty-nine records reconciled total and state durations within approximately 1 ms; seven residuals exposed the absence of explicit session-boundary timestamps. The evaluation concerns architectural behavior rather than training effectiveness; no learning-effect or condition-comparison claim is made. The contribution is an event-driven architecture for inspectable procedural control, recoverable invalid actions, state-level evidence, and decoupled physical-data supervision.

**Index Terms**—virtual reality, smart factory, procedural training, digital twin, state machine, interaction validation, telemetry

## I. INTRODUCTION

Smart manufacturing changes the knowledge required from human operators. In addition to understanding a production sequence, an operator may need to interpret digital guidance, manipulate physical resources, and respond to changes in equipment state. Conventional demonstrations and written procedures remain useful, but they are difficult to repeat under identical conditions and normally provide little information about where execution failed. A VR trainer can provide repeatable practice and record recoverable mistakes before comparable errors carry physical consequences.

Immersive virtual reality (VR) can reproduce an industrial workspace without interrupting production or exposing trainees to equipment hazards. However, a rendered three-dimensional scene is not by itself a training system. A technically useful platform must represent the procedure explicitly, constrain or interpret interaction, vary assistance, and record what happened during execution. Digital-twin research adds a second requirement: the virtual environment should have a defined data boundary with the physical system instead of being described as a twin solely because it resembles a factory [1], [2].

This paper addresses the following research question: *How can an event-driven VR architecture represent, validate, and record a manual manufacturing procedure while remaining operational when an optional read-only factory-data service is unavailable?* The implementation is grounded in a manual labeling workstation within the Smart Factory Connected (SFC) line at HESTIM. The workstation was reconstructed in Unity 6 and deployed on Meta Quest 3.

The paper makes four contributions:

- A requirement-driven architecture separating XR interaction, procedural control, guidance, telemetry, and read-only supervision.
- An event-driven state model that binds each expected action to an inspectable validation rule and state-level record.
- A guidance policy in which cues can be removed without changing completion rules, while invalid actions remain recoverable and observable as counts.
- A decoupled ESP32–FastAPI–Unity path that introduces physical context without adding an XR-to-actuator route.

The scientific claim is architectural: the artifact realizes six requirements and exposes evidence for their functional verification. Session data demonstrate state-level observability; they are not used to infer a training effect.

## II. RELATED WORK

### A. Immersive Training for Manufacturing

Recent reviews identify extended reality (XR) as a medium for manufacturing instruction, assembly support, maintenance, and operator familiarization [3]–[5]. Immersive systems permit safe repetition and embodied interaction, while spatial cues and direct manipulation can support procedural understanding [6], [7]. Reviews also report substantial variation in study design, session reporting, and outcome measures [8]. This variation motivates explicit control and instrumentation when scenarios must be reproducible.

The quality of an XR experience also depends on whether system responses are congruent with user actions and plausible within the represented task [9]. In an industrial procedure, congruence requires more than realistic graphics. Selection and placement should be accepted only when those operations

are meaningful in the current task state. The present work therefore treats expected-action validation as a first-class runtime component.

### B. Digital Twins and Human-Centered Manufacturing

Digital twins are commonly characterized by an identifiable physical entity, a virtual representation, and data connections supporting a defined lifecycle purpose [1], [10]. Modeling, synchronization, communication, and human interaction are therefore central concerns [11], [12]. ISO 23247 structures manufacturing digital twins around observable manufacturing elements and information exchange [13]. Recent systems range from connected learning ecosystems using industrial data [14] to VR/MR training coupled with process simulation [15].

The current platform does not claim closed-loop control. It implements unidirectional supervision and a separate off-line trainer, avoiding conflation of a static model, a digital shadow, and a bidirectional operational twin. Recoverable invalid actions and removable guidance are human-supportive design choices, not evidence of improved learning or well-being. This restrained interpretation follows work on human-centric manufacturing and the need to define human-centricity explicitly [16]–[20].

### C. Positioning and System Gap

Prior work establishes educational potential, connected industrial knowledge frameworks, and simulation-coupled training. The narrower engineering gap addressed here is the inspectability of a workstation-scale runtime: task definitions, semantic XR events, validation rules, guidance, telemetry, and optional physical data require explicit boundaries. Table I distinguishes this contribution from the nearest categories of work.

TABLE I  
POSITIONING RELATIVE TO RECENT WORK

| Work                  | Primary emphasis                                      | Distinction in this paper                                                                 |
|-----------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------|
| XR reviews [4], [5]   | Application scope and evidence synthesis              | Runtime procedure interfaces are the artifact under study.                                |
| Jalilvand et al. [15] | Simulation-coupled VR/MR training and user comparison | Explicit action validators and state-level evidence are central.                          |
| García et al. [14]    | Connected industrial data and human-machine knowledge | The trainer remains standalone; supervision is read-only and optional.                    |
| This work             | Event-driven VR procedure architecture                | Control, validation, guidance, telemetry, and supervision boundaries are jointly exposed. |

### D. Design-Science Method

The work follows an artifact-oriented design-science sequence [21]. The physical workflow was decomposed into objects, actions, and completion conditions, then translated into six requirements and implemented in Unity. Verification combined controller inspection with functional checks: valid and invalid interactions, guided and autonomous execution,

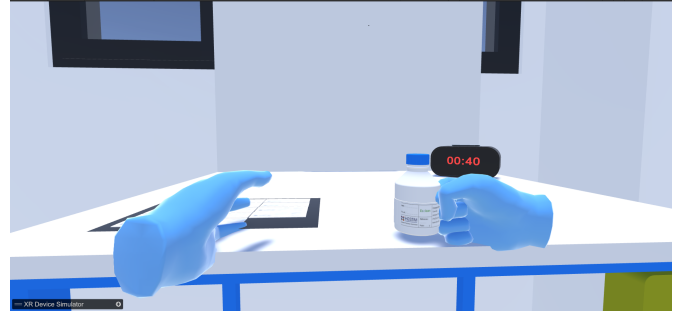


Fig. 1. Operator view of the immersive labeling workstation. The virtual hands, bottle, label, timer, and spatial work area form the human-facing layer of the system.

CSV export, execution without the physical-data service, and inspection of the unidirectional supervision route. Table II reports the evidence used for each requirement. These checks establish implemented behavior, not comparative learning effectiveness or network performance.

## III. CONTEXT AND DESIGN REQUIREMENTS

### A. Labeling Workstation

The project was developed during an engineering internship involving FABLAB at HESTIM Engineering & Business School and ENSAM, Hassan II University of Casablanca. The case study uses the laboratory-scale Smart Factory Connected line at HESTIM. The line represents the production and processing of high-density polyethylene bottles through extrusion-blow molding, trimming, filling, capping, labeling, quality control, packaging, and storage. Labeling is a manual workstation and therefore provides a compact but representative human-in-the-loop procedure.

The user-facing procedure contains six principal actions: positioning an empty bin, placing it in the designated zone, supplying the workstation with a second bin, obtaining a label, selecting a bottle, and applying the label to the correct face. The Unity implementation decomposes pickup and placement checks into nine internal states. This finer decomposition permits the system to time and validate transitions that are visually presented to the trainee as one operation.

Fig. 2 shows the editor-side representation of this decomposition. Each serialized state binds procedural content to a completion condition, allowing the scenario designer to inspect the task sequence without modifying the controller code.

### B. Requirements

Table II links each requirement to an implemented mechanism and functional observation. The requirements were selected to make the trainer inspectable at runtime. Each one corresponds to a responsibility that must remain explicit in an industrial procedure: preserving task order, rejecting premature actions, separating assistance from validation, protecting participant data, tolerating missing supervision data, and preventing immersive interactions from becoming machine-control commands.

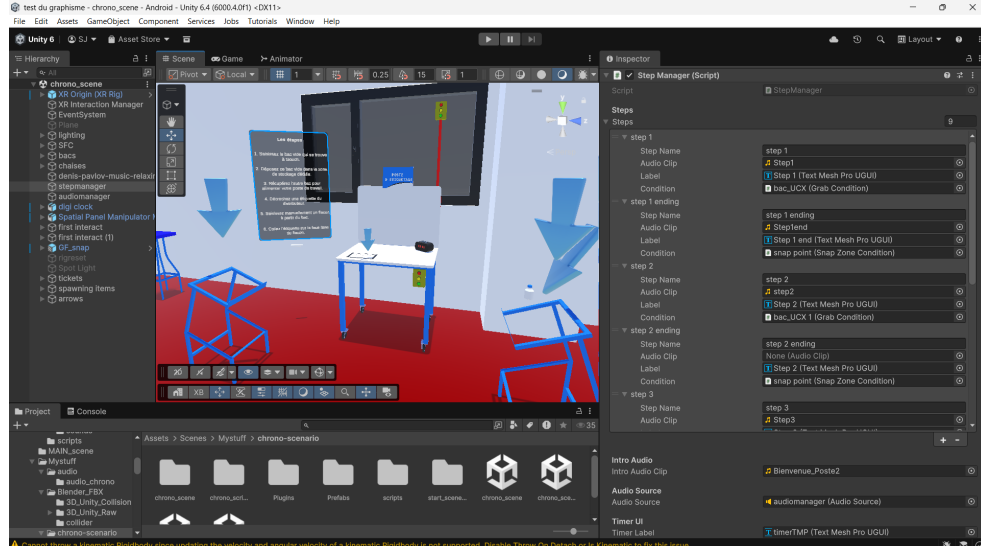


Fig. 2. Editor-side implementation of the labeling procedure. Each serialized state exposes its instruction, audio cue, interface label, and completion condition, linking the workstation decomposition to the runtime controller.

TABLE II  
REQUIREMENT VERIFICATION

| ID | Mechanism                | Functional evidence                                                                               |
|----|--------------------------|---------------------------------------------------------------------------------------------------|
| R1 | Ordered state controller | Valid execution traversed all nine configured states and closed the session.                      |
| R2 | Rule-based validators    | Expected actions advanced the state; unexpected grabs left it active and incremented its counter. |
| R3 | Guidance policy          | Guided and autonomous runs used identical state identifiers and logging fields.                   |
| R4 | Pseudonymous export      | The publication copy removed names, demographics, and exact timestamps.                           |
| R5 | Optional supervision     | The labeling scenario executed while the physical-data service was unavailable.                   |
| R6 | Read-only boundary       | Inspection found an ESP32-to-Unity data route and no XR-to-actuator route.                        |

## IV. PLATFORM ARCHITECTURE

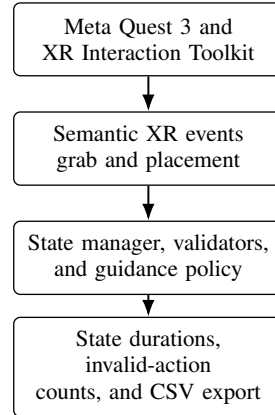
### A. Layered Organization

Fig. 3 separates two runtime paths. The standalone training path converts XR input into semantic events, evaluates them against the active state, and records aggregate evidence. The optional supervision path transports a physical measurement through a mediation service to a read-only Unity display. No dependency arrow connects supervision availability to procedural execution.

### B. XR Interaction Layer

The Unity 6 scene uses the XR Interaction Toolkit and contains an XR Origin, an XR Interaction Manager, interactive bottles and bins, snap zones, spatial panels, audio sources, and task-specific animation. Controller or hand interaction is converted into selection and placement events by the XR layer. The procedure controller does not interpret low-level pose data. It receives semantically meaningful events such

### Standalone training path



### Optional read-only supervision

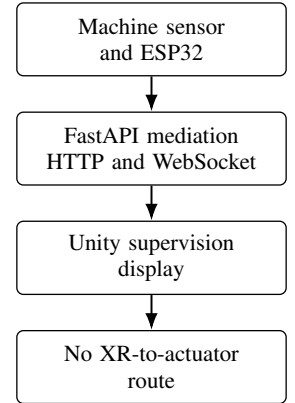


Fig. 3. Runtime responsibility boundaries. Training and read-only supervision are separate paths; loss of the optional service does not interrupt procedural execution.

as “object grabbed” or “object entered target zone.” This boundary follows the portability goal of OpenXR, which provides a common application interface across conformant XR runtimes [22].

Teleportation and turning support movement in the scene, but locomotion does not advance the procedure. State progression occurs only when a registered completion condition is satisfied. Consequently, exploration and accidental contact can be distinguished from task completion.

### C. Procedural State Machine

Let the internal task model be an ordered state set

$$\mathcal{S} = \{s_1, s_2, \dots, s_n\}, \quad n = 9, \quad (1)$$

where a state is represented conceptually as

$$s_i = \langle l_i, a_i, c_i, g_i \rangle. \quad (2)$$

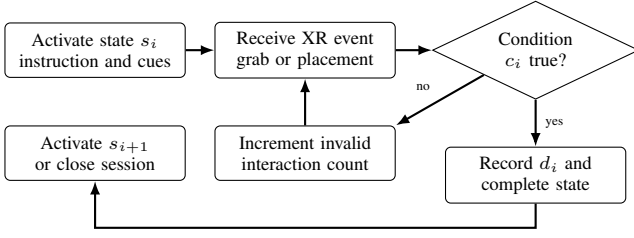


Fig. 4. State transition and interaction-validation logic. Invalid actions are observable but do not advance the procedure.

Here,  $l_i$  is the displayed instruction,  $a_i$  is an optional audio clip,  $c_i$  is the completion condition, and  $g_i$  identifies associated guidance objects. These fields correspond to the serialized configuration visible in Fig. 2. The state becomes active at time  $\tau_i^a$  and completes at  $\tau_i^c$ , producing duration  $d_i = \tau_i^c - \tau_i^a$ .

Fig. 4 shows the runtime logic. On activation, the controller updates the instruction panel and cues. A valid interaction completes the state, records its duration, deactivates its cues, and activates the next state. An invalid grab leaves the state unchanged and increments its error counter. The final state closes and exports the session.

#### D. Completion Conditions and Guidance

Two condition families are used in the labeling implementation. A grab condition verifies that the expected object has been selected during the active state. A snap-zone condition verifies placement in a designated spatial region. The final label-placement condition also triggers the application animation. Keeping the condition as a configurable state field allows different task objects to reuse the same controller logic.

Guidance is implemented as a policy over the state definition. In guided mode, the active instruction, audio clip, and spatial indicators are enabled. In autonomous mode, the procedure and validators remain active while assistance is suppressed. A perturbed mode can introduce scene constraints without changing the logging contract. Thus, guidance changes the available cues while the configured completion rules and evidence fields remain unchanged.

#### E. Telemetry Pipeline

When a session starts, the recorder creates a session context and initializes timers and error counters. Each state transition appends its duration. An interaction involving an unexpected object increments the active state's invalid-grab counter. At completion, the recorder exports the structure

$$R = \langle p, t, m, T, \{d_i\}_1^n, \{e_i\}_1^n \rangle, \quad (3)$$

where  $p$  is a session identifier,  $t$  is the session timestamp,  $m$  is the scene or training mode,  $T$  is total duration, and  $e_i$  is the invalid-interaction count for state  $i$ .

The runtime CSV contains operational metadata, total duration, nine step-duration fields, and nine invalid-grab-count fields. It is an aggregate session record rather than an immutable event stream: individual invalid actions are counted

but not timestamped or identified. The publication copy excludes names, demographic attributes, and exact timestamps, and replaces identity fields with generated session identifiers. The confidential source file remains separate. All participants provided written informed consent.

### V. DIGITAL-TWIN-READY SUPERVISION

#### A. Communication Path

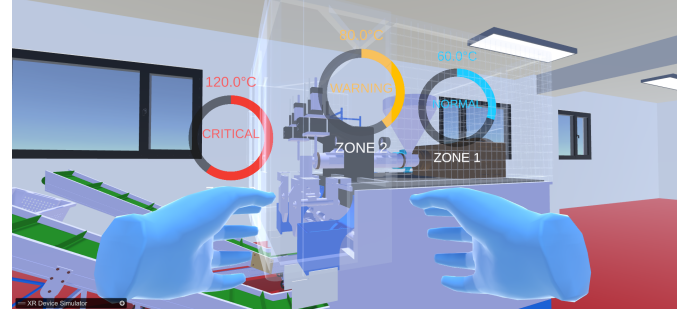
The connected component validates a physical-to-digital path using a temperature measurement from the extrusion-blow-molding machine. An ESP32 sends the value over the local wireless network. FastAPI was selected because one Python service could validate incoming data and expose HTTP and WebSocket interfaces; no comparative framework benchmark was performed. HTTP supports ingestion and snapshot retrieval, while WebSocket pushes updates without application-level polling by the Unity client. Unity therefore depends on the mediation interface rather than the ESP32 message format. The measurement provides line-level supervision and does not alter labeling-state progression. Fig. 5 documents the implemented HTTP request path and the resulting supervisory view used during integration.

```

INFO: 172.16.0.101:8441 - "GET /extrusion-soufflage/latest HTTP/1.1" 200 OK
INFO: 172.16.0.101:8445 - "GET /extrusion-soufflage/latest HTTP/1.1" 200 OK
INFO: 172.16.0.101:8445 - "GET /extrusion-soufflage/latest HTTP/1.1" 200 OK
INFO: 172.16.0.101:8445 - "GET /extrusion-soufflage/latest HTTP/1.1" 200 OK
INFO: 172.16.0.101:8445 - "GET /extrusion-soufflage/latest HTTP/1.1" 200 OK
INFO: 172.16.0.101:8445 - "GET /extrusion-soufflage/latest HTTP/1.1" 200 OK
INFO: 172.16.0.101:8445 - "GET /extrusion-soufflage/latest HTTP/1.1" 200 OK

```

(a) FastAPI request trace



(b) Unity temperature-supervision view

Fig. 5. Runtime evidence of the supervision path: (a) the FastAPI service returned HTTP 200 responses to successive snapshot requests; and (b) Unity presented temperature zones in the immersive interface. The screenshots demonstrate integration, not synchronized latency or reliability.

#### B. Scope and Failure Boundary

This connection is deliberately separated from the labeling trainer. The implemented training scenario operates when the service is unavailable. A production supervision interface would additionally need explicit fresh, stale, and disconnected states; those behaviors were not benchmarked here. The current path is unidirectional and sends no control command to production equipment. It is therefore a read-only supervision baseline rather than a bidirectional operational twin.

The boundary also limits safety risk. FastAPI mediates data formats and communication, while Unity remains a visualization and training client. No path from an XR interaction to a machine actuator is provided.

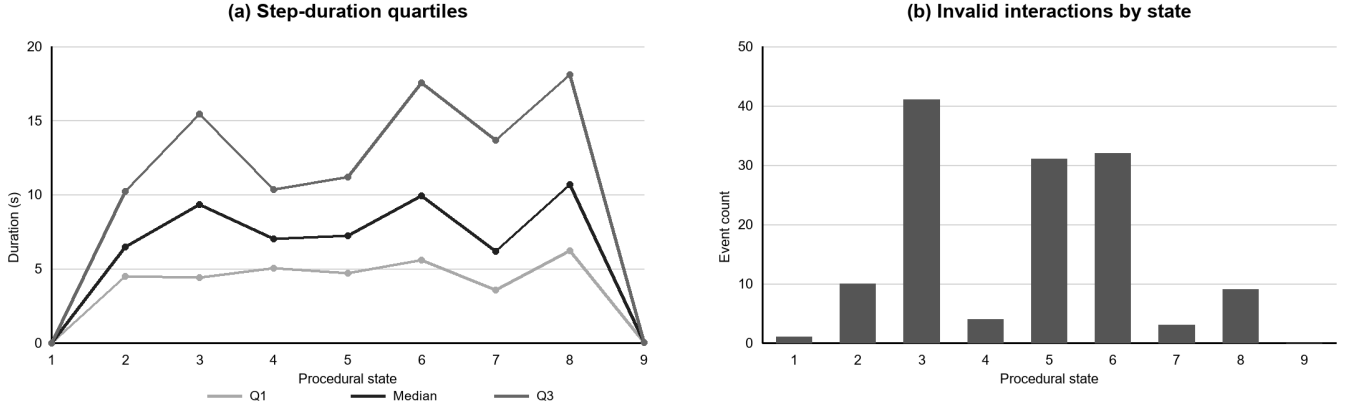


Fig. 6. State-level telemetry from 36 retained sessions: (a) first quartile, median, and third quartile of duration by procedural state; and (b) invalid-interaction counts by state.

## VI. TECHNICAL EVALUATION

### A. Prototype Configuration

The prototype was developed in Unity 6 with the XR Interaction Toolkit, an XR Origin, and an XR Interaction Manager. It targets a Meta Quest 3 headset. The labeling scene includes interactive bins, bottles, labels, placement zones, world-space instruction panels, arrows, audio guidance, a timer, and a result view. Fig. 1 shows the trainee’s workspace, while Fig. 2 shows the editor-side procedure configuration.

The scenario was exercised in guided, autonomous, and perturbed configurations. These modes use the same state identifiers and recorder. Consequently, changing the guidance policy does not require separate logging code or a different data schema.

### B. Measured Results

The source export contained 39 rows because corrected records for one participant had been appended rather than replacing the earlier rows. Processing collapsed exact duplicates and retained the latest timestamp for each participant-condition pair; retained and superseded source-row identifiers were stored for audit. This yielded 36 records from 12 consenting students: seven in Industrial Engineering (GI) and five in Computer Engineering (GIN). All retained records populated the scene identifier, total duration, nine state-duration fields, and nine state-level counters. The counters contained 131 invalid interactions across 31 sessions, showing that rejected actions were represented in the aggregate export.

Recorded session duration ranged from 19.097 to 180.286 s, with a mean of 78.056 s and a median of 71.972 s. The median absolute difference between total duration and summed state durations was below 0.001 s, and 29 of 36 records reconciled within approximately 1 ms. Seven larger residuals ranged from  $-0.515$  to  $+14.990$  s; these raised the mean absolute difference to 0.986 s. Without explicit session-boundary event timestamps, the residuals cannot be assigned to loading, calibration, or a particular transition. State progression is rule-governed, but exact temporal attribution requires timestamped

session-start, state-transition, invalid-action, and session-end events. No hypothesis test, condition comparison, or learning-effect claim is made.

Fig. 6 resolves the same records by procedural state. States 6 and 8 have the largest upper quartiles. Invalid interactions are concentrated in states 3, 5, and 6, which account for 104 of 131 events (79.4%). The state-specific fields expose where validator counts accumulate, but the aggregate records do not reconstruct individual action sequences. The pattern is not interpreted as evidence of participant learning.

### C. Architectural Value

The demonstration supports three technical observations. The serialized state list exposes sequence and completion rules outside scene callbacks. Validators separate generic XR interaction from task-specific acceptance. Finally, controller transitions and telemetry fields share state identifiers, enabling state-level comparison between configured rules and exported aggregates. Event-level traceability would require the additional timestamps identified above.

## VII. DISCUSSION

### A. Scientific and Engineering Implications

The architecture makes procedural training inspectable at the system level. A scenario can be checked to verify that every state has a completion rule, every invalid action has a defined consequence, and every transition has a corresponding record. The XR rig, recorder, guidance service, and condition types are separated from task-specific scene objects and state sequences.

The communication boundary supports incremental extension because Unity consumes mediated updates rather than communicating directly with embedded hardware. Separating this path from the state manager preserved training availability when the service was absent. This is a functional dependency result, not evidence that the connected display is immune to wireless jitter. Packet loss, update latency, reconnection

time, multi-client behavior, headset energy cost, and stale-data presentation were not benchmarked.

### B. Human-Centered Implications

Invalid grabs increment a state-level counter without terminating the session. Guidance can be removed without changing completion rules, and loss of the optional service does not remove access to the trainer. These are human-supportive design choices; the present technical evaluation does not show improved learning, well-being, or autonomy. Trainer-facing records should remain pseudonymous and complement rather than replace instructor judgment.

### C. Limitations

The architecture has been evaluated for one manual workstation, so transfer to maintenance or safety procedures remains unverified. The logger stores aggregate session fields rather than an immutable event stream. Exact XR package, FastAPI, Python, and ESP32 firmware patch versions were not archived, which limits replication. The connected path is unidirectional and was assessed for structural decoupling, not communication performance. Finally, the 36 records demonstrate observability only and do not show improved learning relative to another method.

## VIII. CONCLUSION

The research question was addressed by separating semantic XR events, state-specific validators, guidance, aggregate telemetry, and optional supervision into explicit runtime responsibilities. Functional checks showed that expected actions advance the configured procedure, invalid grabs remain recoverable and counted, guidance can change without changing completion rules, and training can run without the read-only service. Thirty-six retained records demonstrated state-level observability while also revealing the need for timestamped boundary events. The ESP32-FastAPI-Unity path adds physical context but neither controls equipment nor constitutes a validated communication benchmark.

## REFERENCES

- [1] D. Jones, C. Snider, A. Nassehi, J. Yon, and B. Hicks, "Characterising the digital twin: A systematic literature review," *CIRP Journal of Manufacturing Science and Technology*, vol. 29, pp. 36–52, 2020.
- [2] A. Rasheed, O. San, and T. Kvamsdal, "Digital twin: Values, challenges and enablers from a modeling perspective," *IEEE Access*, vol. 8, pp. 21 980–22 012, 2020.
- [3] S. Doolani, C. Wessels, V. Kanal, C. Sevastopoulos, A. Jaiswal, H. Nambiappan, and F. Makedon, "A review of extended reality (XR) technologies for manufacturing training," *Technologies*, vol. 8, no. 4, p. 77, 2020.
- [4] S. Garcia Fracaró, J. Glassey, K. Bernaerts, and M. Wilk, "Immersive technologies for the training of operators in the process industry: A systematic literature review," *Computers & Chemical Engineering*, vol. 160, p. 107691, 2022.
- [5] B. Wang, L. Zheng, Y. Wang, W. Fang, and L. Wang, "Towards the industry 5.0 frontier: Review and prospect of XR in product assembly," *Journal of Manufacturing Systems*, vol. 74, pp. 777–811, 2024.
- [6] J. Radianti, T. A. Majchrzak, J. Fromm, and I. Wohlgenannt, "A systematic review of immersive virtual reality applications for higher education: Design elements, lessons learned, and research agenda," *Computers & Education*, vol. 147, p. 103778, 2020.
- [7] M. Hu, X. Luo, J. Chen, Y. C. Lee, Y. Zhou, and D. Wu, "Virtual reality: A survey of enabling technologies and its applications in IoT," *Journal of Network and Computer Applications*, vol. 178, p. 102970, 2021.
- [8] P. Strojny and N. Duzmanska-Misiarczyk, "Measuring the effectiveness of virtual training: A systematic review," *Computers & Education: X Reality*, vol. 2, p. 100006, 2023.
- [9] M. E. Latoschik and C. Wienrich, "Congruence and plausibility, not presence?! pivotal conditions for XR experiences and effects, a novel model," *Frontiers in Virtual Reality*, vol. 2, 2021.
- [10] A. Fuller, Z. Fan, C. Day, and C. Barlow, "Digital twin: Enabling technologies, challenges and open research," *IEEE Access*, vol. 8, pp. 108 952–108 971, 2020.
- [11] A. Thelen, X. Zhang, O. Fink, Y. Lu, S. Ghosh, B. D. Youn, M. D. Todd, S. Mahadevan, C. Hu, and Z. Hu, "A comprehensive review of digital twin—part 1: Modeling and twinning enabling technologies," *Structural and Multidisciplinary Optimization*, vol. 65, no. 12, p. 354, 2022.
- [12] J. Wilhelm, C. Petzoldt, T. Beinke, and M. Freitag, "Review of digital twin-based interaction in smart manufacturing: Enabling cyber-physical systems for human-machine interaction," *International Journal of Computer Integrated Manufacturing*, vol. 34, no. 10, pp. 1031–1048, 2021.
- [13] *Automation Systems and Integration—Digital Twin Framework for Manufacturing—Part 1: Overview and General Principles*, International Organization for Standardization Std. ISO 23 247-1:2021, 2021.
- [14] Á. García, A. Bregón, and M. A. Martínez-Prieto, "Digital twin learning ecosystem: A cyber-physical framework to integrate human-machine knowledge in traditional manufacturing," *Internet of Things*, vol. 25, p. 101094, 2024.
- [15] I. Jalilvand, J. Jang, B. Gopaluni, and A. S. Milani, "VR/MR systems integrated with heat transfer simulation for training of thermoforming: A multicriteria decision-making user study," *Journal of Manufacturing Systems*, vol. 72, pp. 338–359, 2024.
- [16] F. Longo, A. Padovano, and S. Umbrello, "Value-oriented and ethical technology engineering in industry 5.0: A human-centric perspective for the design of the factory of the future," *Applied Sciences*, vol. 10, no. 12, p. 4182, 2020.
- [17] Y. Lu, H. Zheng, S. Chand, W. Xia, Z. Liu, X. Xu, L. Wang, Z. Qin, and J. Bao, "Outlook on human-centric manufacturing towards industry 5.0," *Journal of Manufacturing Systems*, vol. 62, pp. 612–627, 2022.
- [18] B. Wang, P. Zheng, Y. Yin, A. Shih, and L. Wang, "Toward human-centric smart manufacturing: A human-cyber-physical systems perspective," *Journal of Manufacturing Systems*, vol. 63, pp. 471–490, 2022.
- [19] X. Li, A. Nassehi, B. Wang, S. J. Hu, and B. I. Epureanu, "Human-centric manufacturing for human-system coevolution in industry 5.0," *CIRP Annals*, vol. 72, no. 1, pp. 393–396, 2023.
- [20] S. Hermawati, R. Correa, M. Mohan, G. Lawson, and R. Houghton, "Defining human-centricity in industry 5.0 and assessing the readiness of ergonomics/human factors communities in the UK," *Ergonomics*, vol. 68, no. 6, pp. 857–876, 2024.
- [21] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [22] Khronos Group, "OpenXR: High-performance access to AR and VR platforms and devices," Open standard, 2024. [Online]. Available: <https://www.khronos.org/openxr/>